Plug in Battery Project

Verfasser:

Marko Petrovic ID: 01527067

Anto Škoro ID: 01607413

Date: 01.03.2026

TU Wien / PR 384.171

Contents

1 Einleitung **3**
1.1 Struktur des Dokuments . . . 3
1.2 Darstellung von Energie- und Datenflüssen . . . 3
1.3 Verweis auf Implementierung . . . 3

2 System Überblick **4**
2.1 Energiefluss im System . . . 4
2.2 Kommunikationsstruktur im System . . . 5

3 Hardware Komponenten **5**
3.1 Photovoltaikmodule . . . 5
3.2 MPPT Laderegler . . . 5
3.3 Batteriespeicher . . . 6
3.4 Wechselrichter . . . 7
3.4.1 Anschlüsse . . . 8
3.5 Victron VE.Bus to USB Interface (MK3-USB) . . . 9
3.6 Stromsensor (CT-Klemme) . . . 10
3.7 ESP32 Steuerungseinheit . . . 11

4 Kommunikations Architektur **12**
4.1 Kommunikation mit dem Wechselrichter . . . 13
4.2 ESP32 als Kommunikationsschnittstelle . . . 13
4.3 Verwendete Kommunikationsprotokolle . . . 13
4.3.1 MQTT . . . 14
4.3.2 Modbus TCP . . . 14
4.3.3 Zigbee . . . 15
4.4 Anbindung an Home Assistant . . . 15

5 Systemkonfiguration **16**
5.1 Physikalische Verbindungen . . . 16
5.1.1 Batterie und Wechselrichter . . . 16
5.1.2 ESP32 und Wechselrichter . . . 17
5.1.3 ESP32 und Home Assistant . . . 17
5.2 Netzwerkkonfiguration . . . 17
5.3 MQTT-Konfiguration . . . 17
5.4 Softwarearchitektur der ESP32-Steuerungseinheit . . . 17
5.4.1 Aufgaben der Steuerungseinheit . . . 17
5.4.2 Kommunikationsprinzip . . . 18
5.4.3 Softwarestruktur . . . 18
5.4.4 Funktionsweise . . . 18
5.5 Zigbee-Konfiguration . . . 18
5.5.1 Zigbee-Adapterkonfiguration . . . 18
5.5.2 Ermittlung des seriellen Ports . . . 19
5.5.3 Konfiguration in Zigbee2MQTT . . . 19
5.5.4 Integration in Zigbee2MQTT . . . 20
5.5.5 Datenfluss . . . 21
5.6 Gesamter Datenfluss . . . 21
5.7 Hinweis zur finalen Implementierung . . . 21
5.8 Hinweise zur Erweiterung . . . 22

6 Debugging und Troubleshooting **22**
6.1 Master-Slave-Konfiguration der Batteriekommunikation 22
6.2 Aktivierung der MQTT-Schnittstelle im Victron-System 23
6.3 Ermittlung der vollständigen MQTT-Topic-Struktur 23
6.4 Initialisierung der MQTT-Datenübertragung (Keep-Alive) 24

1 Einleitung

Dieses Dokument dient als technische Dokumentation und als begleitendes Handbuch für das entwickelte Plug-In-Battery-System. Ziel ist es, einen strukturierten Überblick über die Architektur, die Funktionsweise sowie die Kommunikationsstrukturen des Systems zu geben.

Das System kombiniert Photovoltaikmodule, einen Laderegler (MPPT), eine Batteriespeichereinheit sowie einen Wechselrichter zur Versorgung des Hausnetzes. Zusätzlich wird eine Steuerungs- und Kommunikationseinheit auf Basis eines ESP32-Mikrocontrollers eingesetzt, um Systemdaten auszulesen und Steuerbefehle über verschiedene Kommunikationsprotokolle zu übertragen.

Die eigentliche Implementierung des Systems, insbesondere die verwendete Software und der Quellcode für die Steuerungseinheit, ist nicht Bestandteil dieses Dokuments. Der vollständige Code sowie weiterführende technische Informationen zur Implementierung sind im zugehörigen GitHub-Repository dokumentiert: `https://github.com/antoskoro96-spec/Plug-In-Battery/tree/main`

1.1 Struktur des Dokuments

Das Dokument ist so aufgebaut, dass zunächst die grundlegende Systemarchitektur vorgestellt wird. Dabei werden die wichtigsten Hardwarekomponenten des Systems sowie deren Verbindung innerhalb des Energiesystems beschrieben. Anschließend werden die Energieflüsse zwischen den einzelnen Systemkomponenten sowie die Kommunikationsstrukturen innerhalb des Systems erläutert. Darauf aufbauend werden die einzelnen Systemkomponenten detaillierter beschrieben, wobei sowohl ihre Funktion im Energiesystem als auch ihre Rolle in der Datenkommunikation betrachtet wird.

1.2 Darstellung von Energie- und Datenflüssen

Zur besseren Verständlichkeit werden in diesem Dokument zwei Arten von Systemflüssen unterschieden:

- **Leistungspfade** beschreiben den physikalischen Energiefluss innerhalb des Systems, beispielsweise von den Photovoltaikmodulen zur Batterie oder vom Wechselrichter zum Hausnetz.
- **Datenpfade** beschreiben den Informationsaustausch zwischen den einzelnen Komponenten, beispielsweise über MQTT oder Modbus TCP.

In den entsprechenden Systemdiagrammen werden diese beiden Arten von Flüssen visuell voneinander getrennt dargestellt, um sowohl die energetischen als auch die informationstechnischen Zusammenhänge klar nachvollziehbar zu machen.

1.3 Verweis auf Implementierung

Die konkrete Implementierung der Steuerungslogik erfolgt über eine separate Softwarekomponente auf einem Mikrocontroller (ESP32). Der vollständige Quellcode sowie weitere technische Details zur Implementierung sind im zugehörigen GitHub-Repository dokumentiert (siehe Kapitel 1).

Dort sind insbesondere folgende Inhalte enthalten:

- Implementierung der Kommunikationsschnittstellen
- MQTT-Integration
- Verarbeitung und Weiterleitung von Systemdaten

2 System Überblick

Dieser Abschnitt beschreibt den grundlegenden Aufbau des entwickelten Plug-In-Battery-Systems. Ziel ist es, die wichtigsten Systemkomponenten sowie deren energetische und informationstechnische Verbindungen darzustellen.

Das System besteht aus mehreren Hardwarekomponenten, die sowohl über elektrische Energiepfade als auch über Kommunikationsschnittstellen miteinander verbunden sind. Während die Energieflüsse den physikalischen Leistungsfluss innerhalb des Systems beschreiben, dienen die Kommunikationsverbindungen dem Austausch von Messwerten und Steuerinformationen. Abbildung 1 zeigt eine vereinfachte Darstellung der Systemarchitektur mit allen relevanten Komponenten sowie deren Leistungs- und Datenpfaden.

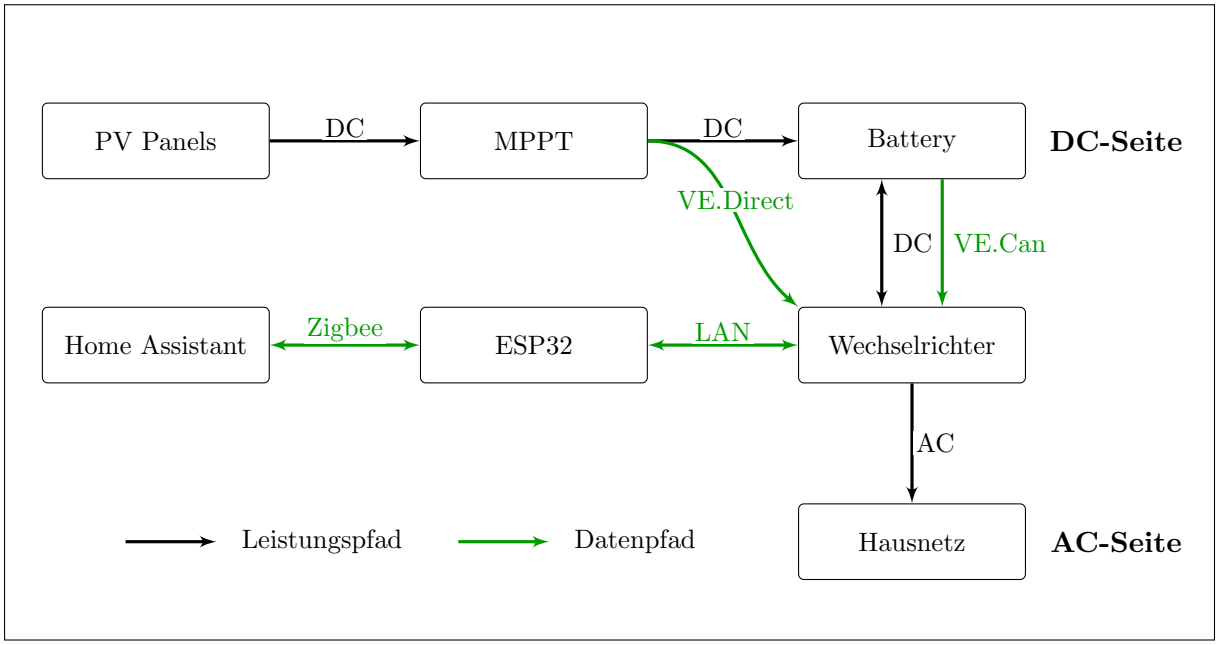

Figure 1: Systemüberblick des Plug-In-Battery-Systems mit Leistungs- und Datenpfaden.

2.1 Energiefluss im System

Der Energiefluss innerhalb des Systems beginnt bei den Photovoltaikmodulen. Die erzeugte elektrische Energie wird zunächst über einen MPPT-Laderegler verarbeitet, der die Leistung der PV-Module an die Ladecharakteristik der Batterie anpasst. Die vom MPPT bereitgestellte Energie wird anschließend zum Laden der Batterie verwendet. Die Batterie dient als Energiespeicher und stellt bei Bedarf elektrische Energie für das System zur Verfügung.

Zwischen Batterie und Wechselrichter besteht eine bidirektionale Verbindung. Dadurch kann der Wechselrichter einerseits Energie aus der Batterie beziehen und andererseits überschüssige Energie in die Batterie zurückspeisen. Der Wechselrichter wandelt die Gleichspannung der Batterie in Wechselspannung um und versorgt damit das Hausnetz. Über das Hausnetz werden schließlich die angeschlossenen Verbraucher mit elektrischer Energie versorgt.

Hinweis zur Systemkonfiguration Die in Abbildung 1 dargestellte Systemarchitektur zeigt eine allgemeine Struktur eines Photovoltaik-Batteriesystems mit PV-Modulen, MPPT-Laderegler, Batteriespeicher und Wechselrichter. Im Rahmen des hier beschriebenen Projekts wurden jedoch nicht alle dargestellten Komponenten physisch integriert oder getestet. Insbesondere die Photovoltaikmodule sowie der MPPT-Laderegler wurden in dieser Arbeit lediglich als mögliche Erweiterung eines Vollständigen Systems berücksichtigt, waren jedoch nicht Bestandteil des prak-

tischen Versuchsaufbaus. Der implementierte und getestete Systemaufbau beschränkt sich auf die Batterie, den Wechselrichter, die Steuerungseinheit auf Basis eines ESP32-Mikrocontrollers sowie die Anbindung an das Hausnetz und die angeschlossenen Verbraucher.

Die Einbindung von PV-Modulen und eines MPPT-Ladereglers ist jedoch in der dargestellten Systemarchitektur vorgesehen und kann in einer erweiterten Systemkonfiguration ohne grundlegende Änderungen der Kommunikationsstruktur integriert werden.

2.2 Kommunikationsstruktur im System

Neben dem Energiefluss existiert innerhalb des Systems eine Kommunikationsstruktur, über die Betriebsdaten der einzelnen Komponenten ausgetauscht werden. Der Wechselrichter fungiert hierbei als zentrale Schnittstelle für Systeminformationen. Sowohl der MPPT-Laderegler als auch das Batteriemanagementsystem der Batterie stellen ihre Messwerte über den Wechselrichter zur Verfügung. Dazu gehören beispielsweise PV-Leistung, Batteriespannung, Batteriestrom sowie der Ladezustand der Batterie.

Die Steuerungs- und Kommunikationseinheit basiert auf einem Mikrocontroller der ESP32-Familie. Dieser übernimmt die Aufgabe, Systemdaten vom Wechselrichter auszulesen sowie Kommunikations- schnittstellen zu externen Systemen bereitzustellen. Über einen Ethernet-Adapter kann der Mikrocontroller eine LAN-Verbindung zum Wechselrichter herstellen, über die Systemdaten ausgelesen und Steuerbefehle übertragen werden. Zusätzlich verfügt der eingesetzte Mikrocontroller über eine integrierte Zigbee-Schnittstelle, über die eine drahtlose Kommunikation mit einem Home-Assistant-System realisiert wird.

Durch diese Struktur können Systemdaten zentral erfasst, ausgewertet und für weiterführende Automatisierungs- oder Visualisierungsaufgaben genutzt werden.

3 Hardware Komponenten

In diesem Kapitel werden die im System eingesetzten Hardwarekomponenten aufgeführt. Für jede Komponente werden die wichtigsten technischen Eigenschaften sowie weiterführende Dokumentationen angegeben.

3.1 Photovoltaikmodule

Die Photovoltaikmodule dienen als Energiequelle des Systems und wandeln einfallende Sonnenstrahlung in elektrische Gleichspannung um.

Wichtige Spezifikationen

- -

Weiterführende Dokumentation

- Herstellerseite: -
- Datenblatt: -

3.2 MPPT Laderegler

Der MPPT-Laderegler übernimmt die optimale Anpassung der Photovoltaikleistung an das Batteriesystem und stellt sicher, dass die maximale verfügbare Leistung der PV-Module genutzt wird.

Figure 2: SmartSolarMPPT

Wichtige Spezifikationen

- Typ: Victron SmartSolar MPPT
- Eingangsspannung: bis zu 150 V
- Maximaler Ladestrom: 35 A
- Kommunikation: VE.Direct / VE.Can

Weiterführende Dokumentation

- Produktseite: `https://www.victronenergy.com`
- Manual: `https://www.nkon.nl/de/amfile/file/download/file/351/product/5707/`
- Datasheet: `https://www.nkon.nl/de/amfile/file/download/file/349/product/5707/`

3.3 Batteriespeicher

Als Energiespeicher wird eine Lithium-Eisenphosphat-Batterie ($LiFePO_4$) eingesetzt. Diese speichert überschüssige Energie und stellt sie bei Bedarf wieder zur Versorgung des Systems zur Verfügung.

Figure 3: Dyness DL5.0C

Wichtige Spezifikationen

- Modell: Dyness DL5.0C
- Batterietyp: $LiFePO_4$
- Nennspannung: 51.2 V
- Kapazität: 100 Ah
- Energieinhalt: 5.12 kWh
- Integriertes Batteriemanagementsystem (BMS)

Weiterführende Dokumentation

- Herstellerseite: `https://dyness.com`
- Manual: `https://www.nkon.nl/de/amfile/file/download/file/11/product/5646/`
- Datasheet: `https://www.nkon.nl/de/amfile/file/download/file/13/product/5646/`
- Victron Example: `https://www.nkon.nl/de/amfile/file/download/file/248/product/5646/`

3.4 Wechselrichter

Der Wechselrichter wandelt die Gleichspannung der Batterie in Wechselspannung um und versorgt damit das Hausnetz. Zusätzlich fungiert er als zentrale Kommunikationsschnittstelle für verschiedene Systemkomponenten.

Figure 4: Wechselrichter Victron MultiPlus-II GX

Wichtige Spezifikationen

- Modell: Victron MultiPlus-II GX

- Nennleistung: 3000 VA
- Batteriespannung: 48 V
- Integrierte Kommunikationsschnittstellen
- Unterstützung von Modbus TCP und MQTT

Weiterführende Dokumentation

- Herstellerseite: `https://www.victronenergy.com`
- Manual: `https://www.nkon.nl/de/amfile/file/download/file/1466/product/5197/`
- Datasheet: `https://www.nkon.nl/de/amfile/file/download/file/380/product/5197/`
- VE.Can RJ45 Abschlusswiderstand: `https://www.victronenergy.de/accessories/ve-can-rj45-terminator`

3.4.1 Anschlüsse

Abbildung 5 zeigt die Anschlussseite des verwendeten Wechselrichters. Über diese Anschlüsse erfolgt sowohl die elektrische Integration in das Energiesystem als auch die Kommunikation mit anderen Systemkomponenten.

Figure 5: Anschlussseite des Wechselrichters Victron MultiPlus-II GX

Im Folgenden werden die wichtigsten Anschlüsse kurz beschrieben:

AC-IN Der AC-IN Anschluss dient zum Anschluss einer externen Wechselspannungsquelle, beispielsweise des öffentlichen Stromnetzes oder eines Generators.

AC-OUT Über die AC-OUT Anschlüsse wird das Hausnetz bzw. die angeschlossenen Verbraucher mit Wechselspannung versorgt.

Batterieanschlüsse (+ -) Die Batterieanschlüsse verbinden den Wechselrichter mit dem Gleichspannungs-Batteriesystem. Über diese Verbindung erfolgt sowohl die Energieentnahme aus der Batterie als auch das Laden der Batterie.

Ethernet Der Ethernet-Anschluss ermöglicht die Integration des Wechselrichters in ein lokales Netzwerk. Über diese Verbindung können Systemdaten ausgelesen und Steuerbefehle übertragen werden. In der beschriebenen Systemarchitektur wird diese Schnittstelle für die Kommunikation mit der ESP32-Steuerungseinheit verwendet.

VE.Bus Die VE.Bus Schnittstelle dient der Kommunikation zwischen Victron Geräten. Sie wird unter anderem für die Verbindung mit zusätzlichen Steuergeräten oder Erweiterungsmodulen verwendet.

VE.Can bzw. BMS-CAN Über die VE.Can-Schnittstelle kann ein externes Batteriemanagementsystem (BMS) mit dem Wechselrichter kommunizieren. Dadurch werden wichtige Batteriedaten wie Ladezustand oder Zellspannungen übertragen. Bei der Dyness-Batterie erfolgt die Verbindung über ein speziell mitgeliefertes VE.Can-Kabel, wobei der VE.Can-Anschluss des Wechselrichters mit dem entsprechenden CAN-Anschluss (IN) der Batterie verbunden wird. Am zweiten VE.Can-Port vom Wechselrichter wird ein Abschlusswiderstand (siehe Abbildung 6) eingesetzt, um eine korrekte Buskommunikation sicherzustellen.

VE.Direct Die VE.Direct Schnittstelle dient zur Kommunikation mit weiteren Victron-Geräten, beispielsweise MPPT-Ladereglern oder Monitoring- Systemen.

USB Der USB-Anschluss ermöglicht Wartungs- und Konfigurationszugriffe auf das Gerät, beispielsweise zur lokalen Parametrierung oder Diagnose.

Figure 6: Victron VE.Can RJ45 Abschlusswiderstand

3.5 Victron VE.Bus to USB Interface (MK3-USB)

Das VE.Bus to USB Interface dient als Kommunikationsadapter zwischen Victron-Geräten mit VE.Bus Schnittstelle und externen Steuerungs- oder Computersystemen. Über dieses Interface können Konfigurations- und Diagnosedaten des Wechselrichters ausgelesen oder Parameter angepasst werden. Das Interface wird in diesem Projekt für die Verbindung zwischen dem Victron-Wechselrichter und einem Computer verwendet, um die neueste Firmware auf den Wechselrichter zu laden und diesen zu konfigurieren.

Figure 7: Victron VE.Bus to USB Interface (MK3-USB)

Wichtige Spezifikationen

- Modell: Victron VE.Bus to USB Interface (MK3-USB)
- Schnittstelle zum Wechselrichter: VE.Bus
- Funktion: Konfiguration, Monitoring und Diagnose

Weiterführende Dokumentation

- Produktseite: `https://www.victronenergy.de/accessories/interface-mk3-usb`
- Configuration Guide: `https://www.victronenergy.com/upload/documents/VictronConnect_configuration_guide_for_VE.Bus_products/15296-VE_Bus_Configuration_guide-pdf-en.pdf`

3.6 Stromsensor (CT-Klemme)

Die CT-Klemme (Current Transformer) dient zur Messung des Wechselstroms in einer Leitung. Der Sensor wird dabei um den entsprechenden Leiter gelegt und ermöglicht eine berührungslose Messung des Stromflusses. In Verbindung mit dem Wechselrichter kann so der Stromfluss im Hausnetz oder am Netzanschlusspunkt erfasst werden. Die gemessenen Werte können anschließend zur Überwachung des Energieflusses oder für Energiemanagementfunktionen verwendet werden.

Figure 8: Stromsensor (CT-Klemme)

Wichtige Spezifikationen

- Typ: CT-Klemme für Victron MultiPlus-II
- Sensortyp: Stromwandler (Current Transformer)

Weiterführende Dokumentation

- Produktseite: `https://www.victronenergy.de/meters-and-sensors/current-transformer-for-multiplus-ii`
- Datasheet: `https://www.victronenergy.com/upload/documents/Datasheet-Current-transformer-DE.pdf`

3.7 ESP32 Steuerungseinheit

Die Steuerungs- und Kommunikationseinheit basiert auf einem Mikrocontroller der ESP32-Familie. Dieser übernimmt die Aufgabe, Systemdaten auszulesen und Kommunikationsschnittstellen zu externen Systemen bereitzustellen.

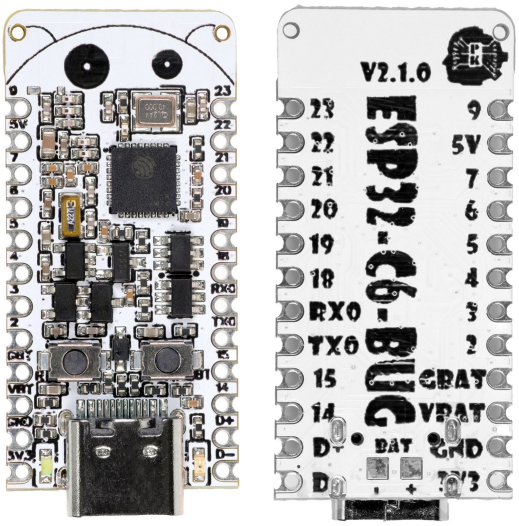

Figure 9: ESP32-C6-BUG

Wichtige Spezifikationen

- Mikrocontroller: ESP32-C6-BUG
- integriertes WLAN
- integriertes Zigbee
- Ethernet-Anbindung über externen Adapter möglich (siehe Abbildung 10)

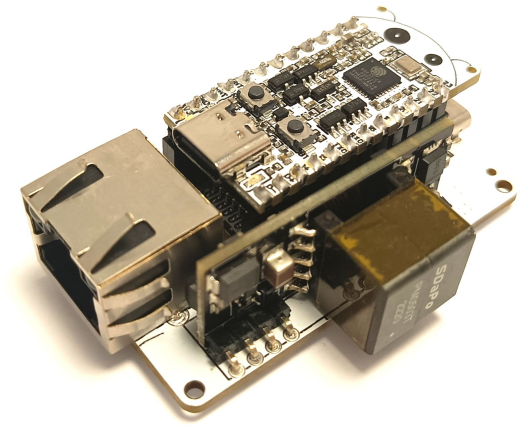

Figure 10: ESP32-C6-BUG LAN-Adapter

Weiterführende Dokumentation

- Hersteller: `https://www.espressif.com`
- Manual: `https://www.mouser.com/catalog/specsheets/Prokyber_esp32c6bugdatasheet.pdf`

4 Kommunikations Architektur

Neben dem Energiefluss zwischen den einzelnen Systemkomponenten existiert innerhalb des Systems eine Kommunikationsstruktur, über die Betriebsdaten erfasst, übertragen und verarbeitet werden können. Abbildung 11 zeigt die vereinfachte Kommunikationsstruktur des Systems. Dargestellt sind die logischen Verbindungen zwischen Wechselrichter, ESP32 und Home-Assistant sowie die dabei verwendeten Kommunikationswege. Diese Kommunikationsarchitektur ermöglicht es, Systemzustände zu überwachen, Messwerte auszulesen und Steuerbefehle zwischen den einzelnen Komponenten zu übertragen.

Die zentrale Rolle innerhalb dieser Struktur übernimmt der Wechselrichter, über den verschiedene Systemdaten verfügbar gemacht werden. Dazu gehören unter anderem Informationen über den Batteriezustand, Energieflüsse im System sowie weitere Betriebsparameter. Der Zugriff auf diese Daten erfolgt über eine zusätzliche Steuerungs- und Kommunikationseinheit auf Basis eines ESP32-Mikrocontrollers.

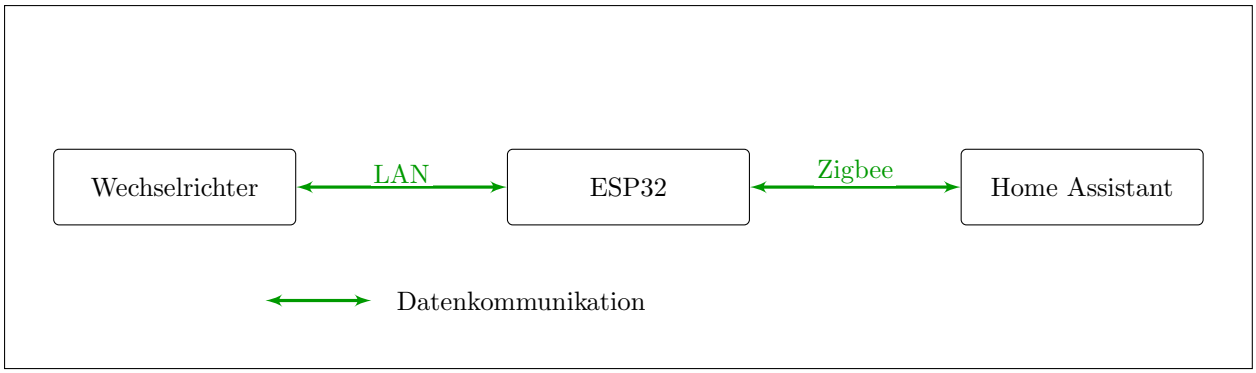

Figure 11: Vereinfachte Kommunikationsübersicht des Systems.

4.1 Kommunikation mit dem Wechselrichter

Der Wechselrichter stellt verschiedene Systemdaten über seine Netzwerkschnittstelle zur Verfügung. In der beschriebenen Systemarchitektur erfolgt die Verbindung zwischen Wechselrichter und ESP32 über eine LAN-Verbindung. Über diese Verbindung kann der ESP32 auf Betriebsdaten des Wechselrichters zugreifen sowie Informationen über den Zustand des Energiesystems auslesen. Zu den verfügbaren Daten gehören unter anderem:

- Batteriedaten (SOC, Strom, Spannung, Leistung) lesen
- Feed-in-Limit schreiben
- Grid-Setpoint schreiben

Weiterführende Dokumentation

- Dokumentation der Victron ESS-Betriebsmodi (Mode 2 und Mode 3) sowie Beschreibung relevanter Regelstrategien: `https://www.victronenergy.com/live/ess:ess_mode_2_and_3`
- Übersicht aller verfügbaren MQTT-Topics und Modbus-Register für Victron-Geräte (inkl. CCGX/GX-Geräte): `https://www.victronenergy.com/upload/documents/CCGX-Modbus-TCP-register-list-3.70.xlsx`

4.2 ESP32 als Kommunikationsschnittstelle

Der ESP32 fungiert innerhalb des Systems als Schnittstelle zwischen dem Energiesystem und externen Anwendungen. Dieser übernimmt die Aufgabe, Systemdaten vom Wechselrichter abzurufen, aufzubereiten und für andere Systeme bereitzustellen. Dadurch wird eine flexible Integration des Energiesystems in weitere Anwendungen ermöglicht.

4.3 Verwendete Kommunikationsprotokolle

Im beschriebenen System kommen mehrere Kommunikationsprotokolle zum Einsatz, die jeweils unterschiedliche Aufgaben innerhalb der Datenübertragung übernehmen. Die Kommunikation zwischen Wechselrichter und ESP32 erfolgt über eine LAN-Verbindung. Dabei werden standardisierte Protokolle verwendet, um Systemdaten auszulesen und Steuerinformationen zu übertragen. Für die drahtlose Anbindung an Home Assistant wird zusätzlich eine Zigbee-Verbindung verwendet. Dadurch kann das Energiesystem in eine übergeordnete Automatisierungs- und Visualisierungsumgebung eingebunden werden. Die in diesem Projekt verwendeten Kommunikationsprotokolle werden im Folgenden kurz beschrieben.

4.3.1 MQTT

MQTT (Message Queuing Telemetry Transport) ist ein leichtgewichtiges Nachrichtenprotokoll, das für die Übertragung von Messwerten und Statusinformationen verwendet wird. Im vorliegenden System wird MQTT genutzt, um Daten des Wechselrichters beziehungsweise der daran angebundenen Komponenten auszulesen. Dazu gehören insbesondere Batteriedaten wie Ladezustand, Spannung, Strom und weitere Betriebswerte des Systems. MQTT eignet sich besonders für die Übertragung zyklisch aktualisierter Systemdaten und stellt damit eine wichtige Grundlage für Monitoring- und Visualisierungsaufgaben dar.

Die von Victron verwendeten MQTT-Topics folgen einer klaren Struktur, bei der der Präfix die Art der Kommunikation definiert:

- **N (Notify):** Übertragung von Messwerten und Zuständen (Read)
- **R (Request):** Anforderung von Daten oder Trigger von Updates
- **W (Write):** Schreiben von Sollwerten und Parametern

Im Rahmen dieses Projekts werden folgende MQTT-Topics verwendet:

Typ	Topic	Beschreibung
N	`N/.../Dc/Battery/Voltage`	Batteriespannung (V)
N	`N/.../Dc/Battery/Current`	Batteriestrom (A)
N	`N/.../Dc/Battery/Power`	Batterieleistung (W)
N	`N/.../Dc/Battery/Soc`	Ladezustand der Batterie (%)
N	`N/.../Settings/CGwacs/AcPowerSetPoint`	Aktueller Power Setpoint (Readback)
N	`N/.../Settings/CGwacs/MaxFeedInPower`	Aktuelle Max Feed-In Power (Readback)
W	`W/.../Settings/CGwacs/AcPowerSetPoint`	Setzen des Power Setpoint
W	`W/.../Settings/CGwacs/MaxFeedInPower`	Setzen der Max Feed-In Power
R	`R/.../system/0/Serial`	Aktivierung der Datenübertragung
R	`R/.../keepalive`	Periodischer Keepalive-Trigger

Table 1: Verwendete MQTT-Topics im System

Die Topics mit dem Präfix N stellen die vom System bereitgestellten Istwerte dar und werden vom ESP32 abonniert. Topics mit dem Präfix W dienen zur Übertragung von Steuerbefehlen, beispielsweise zur Vorgabe eines neuen Netzleistungs-Sollwerts. Die Präfixe R werden verwendet, um Daten aktiv anzufordern oder die zyklische Übertragung zu aktivieren.

4.3.2 Modbus TCP

Modbus TCP ist ein weit verbreitetes Industrieprotokoll zur strukturierten Übertragung von Prozess- und Gerätedaten über ein Netzwerk. Durch die klare Registerstruktur eignet sich Modbus TCP insbesondere für den zuverlässigen Zugriff auf definierte Systemparameter.

Im Rahmen dieses Projekts wurde jedoch bewusst auf eine primäre Nutzung von Modbus TCP verzichtet und stattdessen MQTT als Hauptschnittstelle gewählt. Der Grund dafür liegt in der besseren Integration von MQTT in moderne IoT- und Smart-Home-Systeme wie Home Assistant. Dennoch stellt Modbus TCP eine wichtige alternative Schnittstelle dar, insbesondere für industrielle Anwendungen.

Typische Modbus-Register, die im Kontext dieses Systems verwendet werden könnten, sind in Tabelle 2 dargestellt.

Register	Beschreibung
840	Batteriespannung (V)
841	Batteriestrom (A)
842	Batterieleistung (W)
843	Ladezustand der Batterie (%)
2700	Power Setpoint (W)
2706	Max Feed-In Power (W)

Table 2: Beispielhafte Modbus-Register des Victron-Systems

Während Modbus TCP vor allem für den direkten Zugriff auf niedrigere Systemebenen geeignet ist, bietet MQTT im vorliegenden System eine höhere Flexibilität sowie eine bessere Integration in übergeordnete Softwareplattformen.

4.3.3 Zigbee

Zigbee ist ein drahtloses Kommunikationsprotokoll, das für energieeffiziente und zuverlässige Verbindungen zwischen Automatisierungs- und Smart-Home-Komponenten entwickelt wurde. In diesem Projekt wird Zigbee für die Kommunikation zwischen dem ESP32 und Home Assistant verwendet. Über diese Verbindung können Systemzustände übertragen und Steuerbefehle aus einer Automatisierungsumgebung an das Energiesystem weitergegeben werden. Die Verwendung von Zigbee ermöglicht eine drahtlose Anbindung des Systems an bestehende Smart-Home-Strukturen.

4.4 Anbindung an Home Assistant

Zur Visualisierung und Steuerung des Systems wird Home-Assistant eingesetzt. Die Kommunikation zwischen Home Assistant und dem ESP32 erfolgt über eine Zigbee-Verbindung. Über diese Verbindung können Systemzustände übertragen sowie Steuerbefehle an das Energiesystem gesendet werden.

Zur Integration der Zigbee-Geräte in Home Assistant wird Zigbee2MQTT verwendet. Damit kann man Zigbee-Geräte über einen Koordinator mit einem MQTT-Broker verbinden. Zigbee2MQTT übernimmt die Übersetzung zwischen Zigbee-Protokoll und MQTT-Nachrichten. Die vom ESP32 bereitgestellten Werte werden in MQTT-Topics veröffentlicht und können von Home Assistant abonniert und visualisiert werden. Umgekehrt können Steuerbefehle aus Home Assistant über MQTT an Zigbee2MQTT gesendet und von dort an das Zigbee-Endgerät weitergeleitet werden. Eine mögliche Umsetzung davon kann man in Abbildung 12 sehen. Durch diese Architektur wird eine flexible Integration ermöglicht, da die Kommunikation vollständig über MQTT realisiert ist und somit leicht erweiterbar bleibt. Die detaillierte Implementierung von Zigbee2MQTT wird in Kapitel 5 beschrieben.

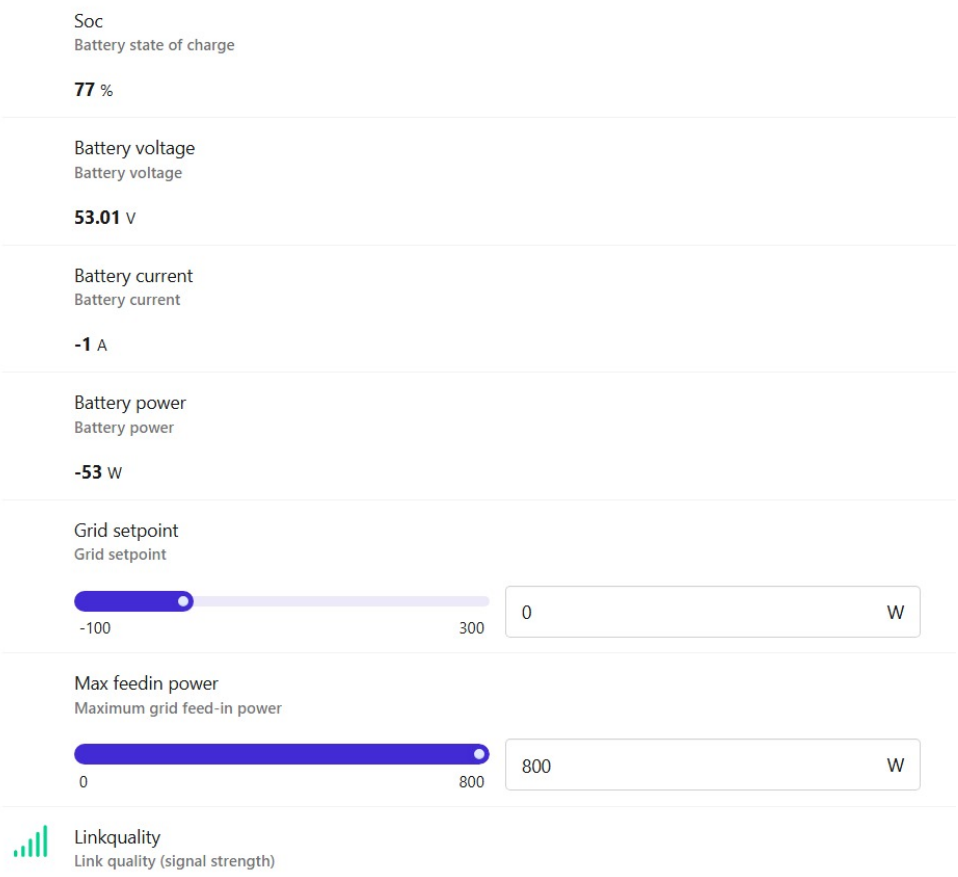

Figure 12: Visualisierung der über Zigbee2MQTT gelesenen und geschriebenen MQTT-Daten.

5 Systemkonfiguration

In diesem Abschnitt wird die konkrete Konfiguration des entwickelten Systems beschrieben. Dabei werden sowohl die physikalischen Verbindungen zwischen den einzelnen Komponenten als auch die verwendeten Kommunikationsschnittstellen und deren Parametrierung erläutert.

Ziel der Konfiguration ist es, eine stabile und reproduzierbare Kommunikation zwischen Batterie, Wechselrichter, ESP32 und Home-Assistant-System sicherzustellen.

5.1 Physikalische Verbindungen

Die physikalische Integration des Systems umfasst sowohl Leistungs- als auch Kommunikationsverbindungen zwischen den einzelnen Komponenten.

5.1.1 Batterie und Wechselrichter

Die Verbindung zwischen Batterie und Wechselrichter erfolgt über zwei separate Schnittstellen:

- **Leistungsanschluss (DC)** zur Energieübertragung
- **Kommunikationsanschluss (VE.Can)** zur Übertragung von BMS-Daten

Die CAN-Kommunikation wird über ein speziell vorgesehenes VE.Can-Kabel realisiert. Dabei wird der VE.Can-Anschluss des Wechselrichters mit dem CAN-IN-Anschluss der Batterie verbunden. Am zweiten VE.Can-Port des Wechselrichters wird ein Abschlusswiderstand eingesetzt, um eine korrekte Buskommunikation sicherzustellen.

5.1.2 ESP32 und Wechselrichter

Die Verbindung zwischen ESP32 und Wechselrichter erfolgt über eine Ethernet-Verbindung. Der ESP32 nutzt dabei einen externen LAN-Adapter, um eine TCP/IP-Verbindung zum lokalen Netzwerk herzustellen. Über diese Verbindung erfolgt der Zugriff auf die vom Wechselrichter beziehungsweise vom GX-System bereitgestellten MQTT-Daten.

5.1.3 ESP32 und Home Assistant

Die Kommunikation zwischen ESP32 und Home Assistant erfolgt drahtlos über Zigbee. Der ESP32 ist dabei in das Zigbee- Netzwerk eingebunden, das über Zigbee2MQTT mit Home Assistant verbunden ist.

5.2 Netzwerkkonfiguration

Für die Kommunikation über MQTT ist eine korrekte Netzwerkkonfiguration erforderlich. Der ESP32 wird in das lokale Netzwerk eingebunden und erhält eine IP-Adresse, entweder statisch oder über DHCP.

Wichtige Parameter sind:

- IP-Adresse des ESP32
- IP-Adresse des Wechselrichters bzw. GX-Geräts
- Adresse des MQTT-Brokers
- Netzwerkport des MQTT-Dienstes

Eine stabile LAN-Verbindung ist Voraussetzung für eine zuverlässige Datenübertragung zwischen Wechselrichter und ESP32.

5.3 MQTT-Konfiguration

Die Kommunikation mit dem Wechselrichter erfolgt im finalen System vollständig über MQTT. Der ESP32 fungiert dabei als MQTT-Client und verbindet sich mit dem vom Victron-System bereitgestellten MQTT-Dienst.

5.4 Softwarearchitektur der ESP32-Steuerungseinheit

Die zentrale Steuerungs- und Kommunikationslogik des Systems wird durch ein speziell entwickeltes Programm auf dem ESP32- Mikrocontroller realisiert. Der ESP32 fungiert dabei als Vermittlungsinstanz zwischen dem Wechselrichter und externen Systemen wie Home Assistant. Er verbindet die verschiedenen Kommunikationsprotokolle und übernimmt die Verarbeitung der Systemdaten.

Die vollständige Implementierung der ESP32-Software ist im zugehörigen GitHub-Repository dokumentiert: `https://github.com/antoskoro96-spec/Plug-In-Battery/tree/main/plug_in_battery`

5.4.1 Aufgaben der Steuerungseinheit

Das auf dem ESP32 ausgeführte Programm übernimmt mehrere zentrale Aufgaben innerhalb des Systems:

- Aufbau der Netzwerkverbindung über Ethernet
- Verbindung mit dem MQTT-System des Victron-GX-Geräts

- Abonnieren und Verarbeiten relevanter MQTT-Topics
- Extraktion und Verarbeitung von Systemdaten
- Bereitstellung der Daten über Zigbee
- Entgegennahme von Steuerbefehlen über Zigbee
- Umsetzung von Steuerbefehlen in MQTT-Schreibzugriffe

5.4.2 Kommunikationsprinzip

Die Software basiert auf einem ereignisgesteuerten Ansatz. Nach dem Aufbau der Netzwerkverbindung verbindet sich der ESP32 mit dem MQTT-Broker des Victron-Systems und abonniert die relevanten Topics. Eingehende MQTT-Nachrichten werden verarbeitet und die enthaltenen Werte extrahiert. Diese Werte werden intern gespeichert und für die Weiterverarbeitung bereitgestellt. Steuerbefehle, die beispielsweise über Home Assistant eingegeben werden, werden über Zigbee empfangen und vom ESP32 in entsprechende MQTT-Nachrichten umgesetzt.

5.4.3 Softwarestruktur

Die Implementierung ist modular aufgebaut und besteht aus mehreren funktionalen Komponenten. Zu den wichtigsten Bestandteilen gehören:

- **Hauptprogramm (.ino):** Initialisierung und Hauptschleife
- **MQTT-Modul:** Verbindung, Subscription und Callback-Handling
- **Zigbee-Modul:** Bereitstellung von Datenpunkten und Empfang von Steuerbefehlen
- **Datenverarbeitung:** Umwandlung und Speicherung der Messwerte

5.4.4 Funktionsweise

Nach dem Aufbau der Netzwerkverbindung verbindet sich der ESP32 mit dem MQTT-System des Wechselrichters und abonniert die für das Projekt relevanten Topics. Eingehende Nachrichten werden analysiert, die enthaltenen Werte werden extrahiert und anschließend intern weiterverarbeitet. Für Steuerfunktionen publiziert der ESP32 gezielt neue Werte auf die vorgesehenen MQTT-Topics. Dadurch können Sollwerte direkt an das Victron-System übergeben werden, ohne dass im finalen Aufbau eine zusätzliche Modbus-TCP-Kommunikation erforderlich ist.

5.5 Zigbee-Konfiguration

Die Zigbee-Kommunikation dient der Anbindung an Home Assistant und ermöglicht sowohl die Visualisierung von Systemdaten als auch die Übertragung von Steuerbefehlen.

5.5.1 Zigbee-Adapterkonfiguration

Für die Kommunikation zwischen ESP32 und Home Assistant über Zigbee ist eine korrekt konfigurierte Zigbee-Koordinator- Hardware erforderlich. Im vorliegenden System wird ein externer Zigbee-USB-Dongle verwendet, der über Zigbee2MQTT in das System integriert ist. Konkret wurde das Modell **SONOFF Zigbee 3.0 USB Dongle Plus** mit dem Chipsatz TI CC2652P und CP2102N eingesetzt. Dieses Gerät fungiert als Zigbee-Koordinator und stellt die zentrale Schnittstelle zwischen den Zigbee-Endgeräten und dem MQTT-basierten System von Home Assistant dar. Der verwendete Dongle unterstützt den Zigbee-Standard in Version 3.0 und ist kompatibel mit verschiedenen Integrationslösungen wie Zigbee2MQTT, ZHA (Zigbee Home Automation) sowie OpenHAB.

5.5.2 Ermittlung des seriellen Ports

Für die Konfiguration des Zigbee-Adapters ist die korrekte Bestimmung des seriellen Ports erforderlich, über den der USB-Dongle mit dem System verbunden ist. In Home Assistant kann dieser Port über die folgenden Schritte ermittelt werden:

- Öffnen der Einstellungen
- Navigieren zu **System**
- Auswahl von **Hardware**
- Öffnen von **Gesamte Hardware**

Dort wird eine Liste aller angeschlossenen Geräte angezeigt. Der Zigbee-Dongle erscheint typischerweise als `ttyUSB` Gerät mit einer eindeutigen Geräte-ID. Ein Beispiel für einen solchen Eintrag ist in Abbildung 13 dargestellt.

Figure 13: Ermittlung des seriellen Ports des Zigbee-Dongles in Home Assistant

Für die weitere Konfiguration sollte bevorzugt der stabile Gerätepfad unter `/dev/serial/by-id/` verwendet werden, da sich dieser im Gegensatz zu `/dev/ttyUSB0` nicht bei Neustarts oder Änderungen der USB-Gerätezuordnung verändert.

5.5.3 Konfiguration in Zigbee2MQTT

Die eigentliche Konfiguration des Adapters erfolgt innerhalb von Zigbee2MQTT. Dabei müssen insbesondere folgende Parameter korrekt gesetzt werden:

- Serieller Port des Zigbee-Dongles
- Verwendeter Adapter-Typ
- Baudrate der seriellen Schnittstelle

Im beschriebenen Aufbau wird der Adapter mit folgenden Einstellungen betrieben:

- **Port:** `/dev/serial/by-id/usb-Itead_Sonoff_Zigbee_3.0_USB_Dongle_Plus_V2_*`
- **Adapter:** `ember`
- **Baudrate:** 115200

Abbildung 14 zeigt die entsprechende Konfiguration innerhalb von Zigbee2MQTT.

Figure 14: Konfiguration des Zigbee-Adapters in Zigbee2MQTT

Eine fehlerhafte oder unvollständige Konfiguration des Zigbee-Adapters führt dazu, dass keine Kommunikation mit den Zigbee-Geräten aufgebaut werden kann. In diesem Fall ist weder eine Datenübertragung noch eine Steuerung über Home Assistant möglich. Die korrekte Adapterkonfiguration stellt somit die Grundlage für die gesamte Zigbee-Kommunikation im System dar.

5.5.4 Integration in Zigbee2MQTT

Der ESP32 wird als Zigbee-Gerät in Zigbee2MQTT eingebunden. Dort werden die verfügbaren Datenpunkte als sogenannte *Entities* dargestellt.

Dazu gehören beispielsweise:

- Ladezustand der Batterie (SoC)
- Batteriespannung
- Batteriestrom
- Batterieleistung
- Grid-Setpoint
- Maximal zulässige Einspeiseleistung (Max Feed-In Power)

Für die Integration in Zigbee2MQTT ist zusätzlich ein gerätespezifischer Converter erforderlich. Dieser definiert die Zuordnung zwischen den Zigbee-Datenpunkten und den in Home Assistant dargestellten Entities. Der im Rahmen dieses Projekts verwendete Converter ist im zugehörigen GitHub-Repository verfügbar: `https://github.com/antoskoro96-spec/Plug-In-Battery`

Der Converter übernimmt insbesondere folgende Aufgaben:

- Zuordnung der Zigbee-Attribute zu lesbaren Entities
- Definition von schreibbaren Parametern
- Umwandlung der Datenformate zwischen Zigbee und MQTT

Zusätzlich ist eine korrekte Konfiguration der Zigbee-Endpoints und der zugehörigen Cluster erforderlich. Dabei werden die verwendeten Datenpunkte als analoge Werte über standardisierte Zigbee-Cluster abgebildet. Für die Umsetzung werden insbesondere folgende Cluster verwendet:

- `genAnalogInput` für lesbare Werte
- `genAnalogOutput` für schreibbare Werte

Die eigentlichen Mess- und Sollwerte werden jeweils über das Attribut `presentValue` innerhalb des entsprechenden Clusters übertragen. Damit ergeben sich folgende Zuordnungen:

- Messwerte (z. B. SoC, Spannung, Strom, Leistung)
  → `genAnalogInput.presentValue`
- Steuerwerte (z. B. Grid-Setpoint, Feed-in Limit)
  → `genAnalogOutput.presentValue`

Damit Änderungen an den Werten zuverlässig erkannt und übertragen werden, ist zusätzlich eine entsprechende Reporting-Konfiguration erforderlich. Diese definiert, in welchen Intervallen oder bei welchen Wertänderungen neue Daten vom Zigbee-Gerät gesendet werden. Durch die Kombination aus Converter, Endpoint-Konfiguration und Cluster-Mapping wird eine vollständige Integration des ESP32 in Zigbee2MQTT und Home Assistant ermöglicht.

5.5.5 Datenfluss

Die Kommunikation erfolgt bidirektional:

- Messwerte werden vom ESP32 an Home Assistant übertragen
- Steuerbefehle werden von Home Assistant über Zigbee an den ESP32 gesendet

Der ESP32 setzt empfangene Steuerbefehle anschließend in entsprechende MQTT-Nachrichten um und überträgt diese an das Victron-System.

5.6 Gesamter Datenfluss

Die vollständige Kommunikationskette im System lässt sich wie folgt zusammenfassen:

- **Lesen von Daten:**
  Batterie → Wechselrichter → MQTT → ESP32 → Zigbee → Home Assistant
- **Steuerung:**
  Home Assistant → Zigbee → ESP32 → MQTT → Wechselrichter

Durch diese Struktur wird eine klare Trennung zwischen Datenerfassung, Weiterverarbeitung und Steuerung erreicht.

5.7 Hinweis zur finalen Implementierung

Während in frühen Konzeptüberlegungen auch eine Ansteuerung über Modbus TCP betrachtet wurde, wurde im finalen aufgebauten und getesteten System die vollständige Kommunikation mit dem Wechselrichter über MQTT realisiert. Damit basiert der praktische Aufbau ausschließlich auf den Kommunikationswegen Ethernet, MQTT und Zigbee.

5.8 Hinweise zur Erweiterung

Das System ist modular aufgebaut und kann um weitere Datenpunkte oder Steuerfunktionen erweitert werden. Für die Erweiterung sind insbesondere folgende Schritte notwendig:

- Ergänzen neuer MQTT-Topics im ESP32-Code
- Implementieren zusätzlicher Verarbeitungslogik
- Erweiterung der Zigbee-Attribute bzw. Exposes

Dadurch kann das System flexibel an neue Anforderungen angepasst werden, ohne die grundlegende Architektur zu verändern.

6 Debugging und Troubleshooting

Während der Entwicklung des Systems traten mehrere typische Probleme auf, die insbesondere aus der Kombination verschiedener Kommunikationsprotokolle (MQTT, Modbus TCP, Zigbee) sowie der verwendeten Hardware (ESP32-C6, Victron-System) resultierten.

In diesem Abschnitt werden die wichtigsten Probleme sowie deren Ursachen und Lösungen beschrieben, um zukünftige Implementierungen zu erleichtern.

6.1 Master-Slave-Konfiguration der Batteriekommunikation

Es gab ein Problem bei der Kommunikation zwischen Batterie und Wechselrichter über die VE.Can-Schnittstelle.

Problem:

- Keine Batteriedaten im System verfügbar
- Der Wechselrichter konnte keine korrekten BMS-Daten auslesen

Ursache:

Die verwendete Dyness DL5.0C benötigt eine korrekte Master-Slave-Konfiguration, welche über DIP-Schalter an der Batterie eingestellt wird. Ist diese Konfiguration falsch, kann keine gültige Kommunikation mit dem Wechselrichter stattfinden.

Lösung: Einstellung der Master-Konfiguration

Die Batterie muss gemäß Herstellerdokumentation als Master konfiguriert werden. Dies erfolgt über die entsprechenden DIP-Schalter an der Batterie wie in Abbildung 15.

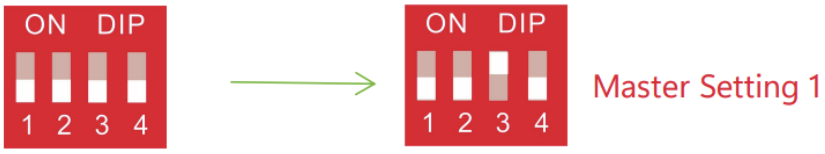

Figure 15: DIP-Schalter Master-Konfiguration.

Zusätzlicher Hinweis:

Die genaue DIP-Schalter-Konfiguration sowie weitere Einstellungen zur Inverter-Kompatibilität sind in der Herstellerdokumentation der Batterie beschrieben: `https://www.nkon.nl/de/amfile/file/download/file/11/product/5646/`

6.2 Aktivierung der MQTT-Schnittstelle im Victron-System

Ein weiteres Problem trat bereits vor der eigentlichen Datenkommunikation auf: Trotz korrekt konfigurierter Netzwerkverbindung konnte keine MQTT-Kommunikation mit dem Victron-System aufgebaut werden.

Problem:

- Der ESP32 konnte keine Daten vom Wechselrichter beziehen

Ursache:

Die MQTT-Schnittstelle ist im Victron-System (Cerbo GX) standardmäßig nicht immer vollständig aktiviert und muss explizit im Systeminterface freigegeben werden. Die Konfiguration erfolgt über das Webinterface (venus.local). Dort befinden sich unter den Service-Einstellungen die relevanten Optionen für MQTT.

Lösungsansatz: Aktivierung der MQTT-Dienste

Im Menü `Services` müssen die Einstellungen wie in Abbildung 16 zu sehen aktiviert werden.

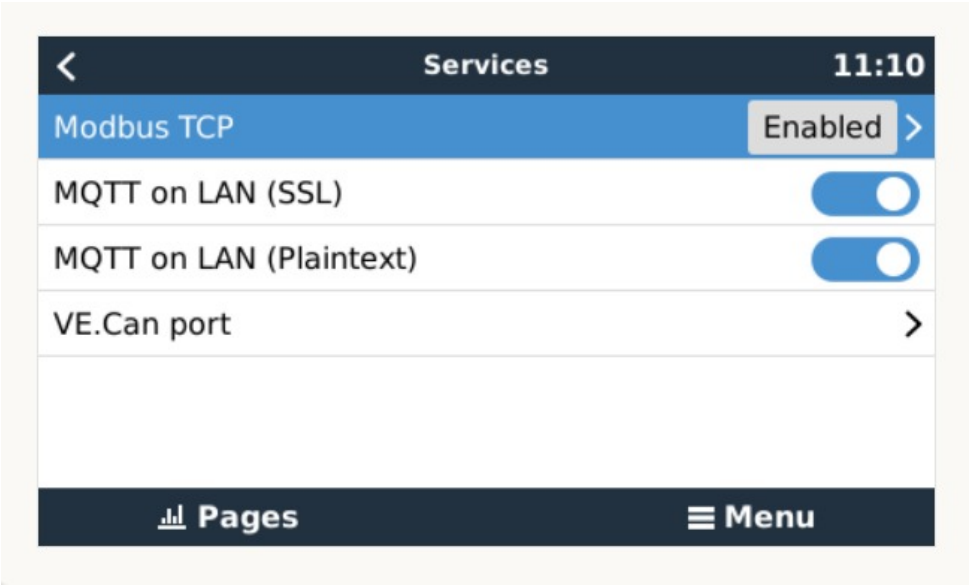

Figure 16: Einstellungen für MQTT im Webinterface vom Wechselrichter.

6.3 Ermittlung der vollständigen MQTT-Topic-Struktur

Ein wesentliches Problem bestand darin, dass die offizielle Victron-Dokumentation (z. B. Excel-Listen der verfügbaren Datenpunkte) zwar einzelne Parameter beschreibt, jedoch nicht den vollständigen MQTT-Topic-Pfad angibt.

Beispielsweise ist in der Dokumentation ein der Topic für einen Wert wie `Voltage` angegeben:

```
/Dc/Battery/Voltage
```

Jedoch benötigt man für den vollständigen Pfad:

```
/N/<device_id>/system/0/Dc/Battery/Voltage
```

Problem:

- Unklarheit über die vollständige Topic-Struktur
- Fehlende Informationen über Device-ID und Zwischenpfade (z. B. `system/0`)

Lösungsansatz: MQTT Explorer

Zur Ermittlung der tatsächlichen Topic-Struktur wurde ein MQTT-Client (z. B. MQTT Explorer) verwendet. Dabei wurde eine Verbindung zum MQTT-Broker des Victron-Systems hergestellt (z. B. Cerbo GX):

- Host: IP-Adresse des Cerbo GX
- Port: 1883
- Protokoll: `mqtt://`

Nach erfolgreicher Verbindung konnte die gesamte Topic-Struktur in Baumform visualisiert werden.

Vorgehensweise:

1. Verbindung zum MQTT-Broker herstellen
2. Navigieren durch die Topic-Struktur (z. B. `N/`)
3. Identifikation der Device-ID (z. B. `48e7da85ccd9`)
4. Schrittweises Öffnen der Unterordner bis zum gewünschten Wert

Beispiel:

- `N/48e7da85ccd9/system/0/Dc/Battery/Voltage`
- `N/48e7da85ccd9/system/0/Dc/Battery/Soc`

6.4 Initialisierung der MQTT-Datenübertragung (Keep-Alive)

Ein weiteres wesentliches Problem trat nach erfolgreicher Verbindung zum MQTT-Broker auf: Es wurden nicht alle erwarteten Topics angezeigt, sondern lediglich ein minimaler Datensatz, wie z. B.:

```
N/<device_id>/keepalive
```

Problem:

- Trotz aktiver Verbindung waren keine vollständigen Systemdaten sichtbar
- Es wurden nur einzelne Topics (z. B. `keepalive`) veröffentlicht

Ursache:

Das Victron-System (Cerbo GX) sendet MQTT-Daten nicht kontinuierlich automatisch, sondern benötigt zunächst eine explizite Anfrage (Keep-Alive Request), um die vollständige Datenübertragung zu starten.

Ohne diese Initialisierung bleibt die Topic-Struktur unvollständig.

Lösungsansatz: Senden eines Keep-Alive Requests

Die Lösung bestand darin, aktiv eine Nachricht an ein entsprechendes Request-Topic zu senden:

```
R/<device_id>/system/0/Serial
```

bzw. allgemein ein Request (`R/`)-Topic des Systems.

Funktionsweise:

- Der ESP32 sendet eine Anfrage an ein `R/`-Topic
- Das Victron-System interpretiert dies als Datenanforderung
- Anschließend beginnt das System, alle relevanten `N/`-Topics zu publizieren